

INFORME TÉCNICO DE INGENIERÍA

Sistema de Posicionamiento con Análisis de Desplazamiento, Velocidad y Monitoreo de Trayectorias

mediante el uso de un brazo robótico y métodos numéricos aplicados

Proyecto: Sistema de Posicionamiento con Análisis de Desplazamiento, Velocidad y Monitoreo de Trayectorias mediante el uso de un brazo robótico.

Área embarcada: Robótica Cinemática, Métodos Numéricos, Adquisición de Datos y Sistemas Embebidos.

Plataformas: Arduino & Octave.

Estudiante: Huaman Marca Angel Joel (23190057)

1. Introducción y Justificación Experimental

1.1 Introducción

En la automatización industrial moderna, la precisión espacial y la repetibilidad de los brazos robóticos articulados son fundamentales para tareas críticas como el ensamblaje de componentes, la soldadura de precisión y el transporte automatizado de materiales. La capacidad de un manipulador para interactuar con su entorno de manera predecible depende directamente de la correcta traducción de las variables articulares (ángulos de los servomotores) a coordenadas del espacio cartesiano tridimensional (X, Y, Z).

Este proyecto aborda el diseño, implementación y validación de un sistema de posicionamiento en lazo abierto mediante un brazo robótico articulado controlado por microcontrolador, integrando herramientas de telemetría asíncrona y procesamiento digital de señales para el análisis de trayectorias en tiempo real. En su versión actual, el proyecto amplía su alcance original —centrado únicamente en la reconstrucción geométrica del recorrido— para incorporar un módulo completo de análisis cinemático numérico, capaz de estimar la velocidad instantánea del efector final y de calcular, mediante integración numérica, la distancia real recorrida durante cada ciclo de operación.

1.2 Justificación Experimental

La experimentación física en robótica suele diferir de los modelos puramente teóricos debido a factores inherentes al hardware, tales como las tolerancias de fabricación, las holguras mecánicas (backlash), el ruido eléctrico en los sensores analógicos de control (joysticks) y la imprecisión en el montaje mecánico de los actuadores.

La justificación de este entorno experimental radica en la necesidad de desarrollar e implementar un algoritmo de Cinemática Directa (Forward Kinematics) adaptado al mundo real, complementado con una etapa de software capaz de adquirir, filtrar y reconstruir geoméricamente la trayectoria espacial descrita por el efector final al trasladar un objeto desde un punto inicial A hasta un punto final B. A esta etapa se suma ahora un componente adicional de Métodos Numéricos orientado a extraer información cinemática de segundo orden —la velocidad— directamente de las coordenadas discretas capturadas, sin necesidad de sensores de velocidad físicos adicionales. La validación del sistema permite cuantificar y corregir discrepancias métricas, así como caracterizar el comportamiento dinámico del brazo, asegurando su viabilidad para aplicaciones didácticas o semi-industriales de posicionamiento analítico.

2. Esquema del Arreglo Experimental y Armado del Brazo

El sistema experimental se compone de un entorno híbrido distribuido en dos capas funcionales: Hardware de Control y Captura (Capa Embebida) y Software de Procesamiento e Historial (Capa Analítica).

2.1 Componentes del Sistema Físico y Conectividad

- Actuadores del Manipulador: Cuatro servomotores independientes encargados de los grados de libertad de la estructura (Cintura, Hombro, Codo y Pinza/Garra) comandados mediante señales de modulación por ancho de pulsos (PWM).
- Unidad de Procesamiento Central: Microcontrolador ATmega328P (Arduino Uno) operando a 16 MHz. Ejecuta un bucle principal multitarea encargado de la lectura analógica de mandos, la interpolación geométrica y la transmisión serial a 9600 bps.
- Módulo de Interfaz de Usuario (HMI): Una pantalla LCD 16x2 controlada mediante el protocolo de comunicación serial I²C para la visualización asíncrona de variables de articulación.
- Módulo de Adquisición de Datos (DAQ): Dos palancas de mando (Joysticks) analógicas de dos ejes cada una para el direccionamiento dinámico de los eslabones, provistas de pulsadores digitales configurados en modo INPUT_PULLUP.

2.2 Guía de Armado Mecánico y Conexiones Eléctricas

El montaje mecánico del brazo robótico articulado de 4 grados de libertad (DOF) debe realizarse de forma secuencial y ascendente (desde la base hasta el efector final):

1. Base y Eje de Cintura: Fijar el chasis o plato giratorio a una superficie estable. Instalar el primer servomotor orientado verticalmente para controlar la rotación sobre el plano horizontal (X-Y). El cuerpo de este eslabón define el origen sobre la mesa.
2. Eslabón del Hombro (L₁): Acoplar mecánicamente el soporte estructural del hombro directamente al piñón del servo de la cintura. Montar el segundo servomotor perpendicular al primero para proveer elevación en el eje vertical (Z). Medir la distancia vertical desde la superficie de apoyo hasta el centro del eje de este servo para establecer la constante L₀_BASE.
3. Eslabón del Codo (L₂): Extender el brazo mecánico instalando la tercera articulación a una distancia estrictamente equivalente a la longitud teórica del eslabón L₁. Montar el tercer servomotor de manera que su eje sea paralelo al del hombro, permitiendo un movimiento coordinado en el plano vertical.
4. Efector Final (Pinzas/Garra): Ensamblar el mecanismo de la pinza en el extremo distal de L₂. El cuarto servomotor se encarga exclusivamente de la apertura y cierre para sujetar el objeto de estudio. La distancia medida desde el eje del codo hasta el centro geométrico de sujeción de la pinza define la constante física L₂.

Componente Físico	Tipo de Señal	Pin en Arduino	Descripción del Control
Servo Cintura	Salida PWM	Pin 8	Rotación de la Base
Servo Hombro	Salida PWM	Pin 6	Elevación del Hombro
Servo Codo	Salida PWM	Pin 10	Articulación del Codo
Servo Pinzas	Salida PWM	Pin 11	Apertura/Cierre de Garra
Joystick 1 (VRY)	Entrada Analógica	Pin A2	Control de Giro de Cintura
Joystick 1 (VRX)	Entrada Analógica	Pin A3	Control de Elevación de Hombro

Componente Físico	Tipo de Señal	Pin en Arduino	Descripción del Control
Joystick 2 (VRX)	Entrada Analógica	Pin A1	Control de Flexión de Codo
Joystick 2 (VRY)	Entrada Analógica	Pin A0	Control Gradual de Pinzas
Botón SW Izq	Entrada Digital	Pin 2 (PULLUP)	Pulsador Punto A (Muestreo ON)
Botón SW Der	Entrada Digital	Pin 4 (PULLUP)	Pulsador Punto B (Muestreo OFF)
Pantalla LCD	Comunicación Bus	SDA (A4) / SCL (A5)	Protocolo I ² C (Dirección 0x27)

3. Modelo Matemático y Cinemática Directa

Para determinar la posición espacial del efector final, se modeló el brazo robótico mediante geometría plana proyectada sobre un sistema de coordenadas cilíndricas, que posteriormente se transforma en un espacio cartesiano euclidiano.

Considerando las longitudes de los eslabones del hombro (L_1), del codo a la garra (L_2), la altura intrínseca de la base fija (L_0_BASE), y los factores de calibración por hardware, las ecuaciones de transformación aplicadas en tiempo real por el firmware son:

3.1 Cálculo de los Ángulos en Radianes (con factores de compensación estática)

$$\theta_cintura_rad = (\theta_cintura + \Delta\theta_cintura) \times (\pi / 180)$$

$$\theta_hombro_rad = (\theta_hombro + \Delta\theta_hombro) \times (\pi / 180)$$

$$\theta_codo_rad = [(\theta_hombro + \Delta\theta_hombro) + (\theta_codo + \Delta\theta_codo) - 90^\circ] \times (\pi / 180)$$

3.2 Proyección del Radio de Alcance Horizontal (r)

$$r = L_1 \times \cos(\theta_hombro_rad) + L_2 \times \cos(\theta_codo_rad)$$

3.3 Conversión a Coordenadas Cartesianas Tridimensionales (X, Y, Z)

$$X = r \times \cos(\theta_cintura_rad)$$

$$Y = r \times \sin(\theta_cintura_rad)$$

$$Z = L_0_BASE + L_1 \times \sin(\theta_hombro_rad) + L_2 \times \sin(\theta_codo_rad)$$

Este modelo geométrico constituye la base sobre la cual se construye toda la etapa analítica posterior: las coordenadas (X, Y, Z) generadas en tiempo real por el firmware son precisamente el insumo discreto que, una vez muestreado a intervalos fijos de tiempo, permite aplicar las técnicas de métodos numéricos descritas en la Sección 5 para estimar velocidad y distancia recorrida.

4. Códigos Fuentes del Sistema

4.1 Código de Control y Adquisición Embebida (Arduino)

Este firmware gestiona la lectura de los joysticks, el cálculo de la cinemática directa en tiempo real y el envío por puerto serial de las coordenadas (X, Y, Z) del efector final mientras el objeto se encuentra sujetado.

```
#include <Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
Servo Cintura; Servo Hombro; Servo Codo; Servo Pinzas;
const float L0_BASE = 6.5; const float L1 = 8.0; const float L2 = 14.0;
const int OFFSET_CINTURA = 0; const int OFFSET_HOMBRO = 0; const int OFFSET_CODO = 0;
int GRADOS_CINTURA = 85; int GRADOS_HOMBRO = 20; int GRADOS_CODO = 115; int GRADOS_PINZAS = 10;
#define VRX_2 A1
#define VRY_2 A0
#define VRY_1 A2
#define VRX_1 A3
#define BOTON_A 2
#define BOTON_B 4
```

```

bool objetoSujetado = false; unsigned long tiempoAnteriorLog = 0;
const unsigned long INTERVALO_MUESTREO = 500; unsigned long tiempoMensajeLcd = 0;
int estadoPantalla = 0; LiquidCrystal_I2C lcd(0x27, 16, 2);

float aRadianes(int grados) { return grados * PI / 180.0; }

void calcularPosicionActual(float &x, float &y, float &z) {
    float radCintura = aRadianes(GRADOS_CINTURA + OFFSET_CINTURA);
    float radHombro = aRadianes(GRADOS_HOMBRO + OFFSET_HOMBRO);
    float radCodo = aRadianes((GRADOS_HOMBRO + OFFSET_HOMBRO) + (GRADOS_CODO + OFFSET_CODO) - 90);
    float r = L1 * cos(radHombro) + L2 * cos(radCodo);
    x = r * cos(radCintura); y = r * sin(radCintura);
    z = L0_BASE + (L1 * sin(radHombro) + L2 * sin(radCodo));
}

void setup() {
    Serial.begin(9600);
    Cintura.attach(8); Hombro.attach(6); Codo.attach(10); Pinzas.attach(11);
    pinMode(BOTON_A, INPUT_PULLUP); pinMode(BOTON_B, INPUT_PULLUP);
    lcd.begin(16, 2); lcd.backlight(); lcd.clear();
}

void loop() {
    unsigned long tiempoActual = millis();
    int LVRY_1 = analogRead(VRY_1); int LVRX_1 = analogRead(VRX_1);
    int LVRX_2 = analogRead(VRX_2); int LVRY_2 = analogRead(VRY_2);
    bool presionandoA = (digitalRead(BOTON_A) == LOW);
    bool presionandoB = (digitalRead(BOTON_B) == LOW);
    if (LVRY_1 < 340) GRADOS_CINTURA -= 1; else if (LVRY_1 > 680) GRADOS_CINTURA += 1;
    GRADOS_CINTURA = constrain(GRADOS_CINTURA, 0, 175);
    if (LVRX_1 < 340) GRADOS_HOMBRO += 1; else if (LVRX_1 > 680) GRADOS_HOMBRO -= 1;
    GRADOS_HOMBRO = constrain(GRADOS_HOMBRO, 10, 150);
    if (LVRX_2 < 340) GRADOS_CODO -= 1; else if (LVRX_2 > 680) GRADOS_CODO += 1;
    GRADOS_CODO = constrain(GRADOS_CODO, 0, 150);
    if (LVRY_2 < 340) GRADOS_PINZAS -= 1; else if (LVRY_2 > 680) GRADOS_PINZAS += 1;
    GRADOS_PINZAS = constrain(GRADOS_PINZAS, 10, 58);
    float currentX, currentY, currentZ;
    calcularPosicionActual(currentX, currentY, currentZ);
    if (presionandoA && !objetoSujetado) {
        objetoSujetado = true; Serial.println("START_A");
        lcd.clear(); lcd.print("Agarrando Objeto");
        estadoPantalla = 1; tiempoMensajeLcd = tiempoActual;
    } else if (presionandoB && objetoSujetado) {
        objetoSujetado = false; Serial.println("STOP_B");
        lcd.clear(); lcd.print("Soltando Objeto");
        estadoPantalla = 2; tiempoMensajeLcd = tiempoActual;
    }
    if (estadoPantalla == 1 && (tiempoActual - tiempoMensajeLcd >= 1000)) { estadoPantalla = 0; lcd.clear(); }
    else if (estadoPantalla == 2 && (tiempoActual - tiempoMensajeLcd >= 2000)) { estadoPantalla = 0; lcd.clear(); }
    if (estadoPantalla == 0) {
        if (objetoSujetado) {
            if (tiempoActual - tiempoAnteriorLog >= INTERVALO_MUESTREO) {
                tiempoAnteriorLog = tiempoActual;
                Serial.print(currentX, 2); Serial.print(",");
                Serial.print(currentY, 2); Serial.print(",");
                Serial.println(currentZ, 2);
            }
            lcd.setCursor(0, 0); lcd.print("Moviendo Objeto");
        } else {
            lcd.setCursor(0, 0); lcd.print("C:"); lcd.print(GRADOS_CINTURA);
            lcd.setCursor(8, 0); lcd.print("H:"); lcd.print(GRADOS_HOMBRO);
        }
    }
}

```

```
}  
Cintura.write(GRADOS_CINTURA); Hombro.write(GRADOS_HOMBRO);  
Codo.write(GRADOS_CODO); Pinzas.write(GRADOS_PINZAS);  
delay(40);  
}
```

4.2 Script de Filtrado y Reconstrucción Gráfica (MATLAB / Octave)

Esta primera versión del script analítico se centró exclusivamente en depurar el ruido de los datos brutos y reconstruir la trayectoria espacial mediante un filtro de promedio móvil (FIR), calculando además la distancia total recorrida por sumatoria directa de segmentos euclidianos.

```
% =====  
% PROCESAMIENTO FILTRADO Y OPTIMIZADO DE TRAYECTORIA  
% =====  
clear; clc; close all;  
  
datos_brutos = [ ... ]; % matriz Nx3 con columnas X, Y, Z  
  
UMBRAL_RUIDO = 0.25; datos_filtrados = datos_brutos(1, :);  
for i = 2:size(datos_brutos, 1)  
    p_ant = datos_filtrados(end, :); p_act = datos_brutos(i, :);  
    if sqrt(sum((p_act - p_ant).^2)) > UMBRAL_RUIDO  
        datos_filtrados = [datos_filtrados; p_act];  
    end  
end  
  
X_raw = datos_filtrados(:, 1); Y_raw = datos_filtrados(:, 2); Z_raw = datos_filtrados(:, 3);  
b = ones(1, 3) / 3; X = filter(b, 1, X_raw); X(1:2) = X_raw(1:2);  
Y = filter(b, 1, Y_raw); Y(1:2) = Y_raw(1:2); Z = filter(b, 1, Z_raw); Z(1:2) = Z_raw(1:2);  
distancia_total = sum(sqrt(diff(X).^2 + diff(Y).^2 + diff(Z).^2));  
  
figure(); plot3(X, Y, Z, '-o', 'LineWidth', 2); grid on; hold on;  
plot3(X(1), Y(1), Z(1), 'gs', 'MarkerFaceColor', 'g');  
plot3(X(end), Y(end), Z(end), 'rd', 'MarkerFaceColor', 'r');  
axis equal; title('Trayectoria Espacial Real del Objeto'); view(60, 25);
```

A partir de esta primera etapa, el proyecto evolucionó hacia una segunda versión del script analítico —detallada en la Sección 4.3— que incorpora explícitamente los temas de Diferenciación Numérica e Integración Numérica propios del curso de Métodos Numéricos, ampliando el alcance del análisis de la simple reconstrucción geométrica a la caracterización dinámica completa del movimiento del brazo.

4.3 Script de Análisis Cinemático de Velocidad y Distancia (Octave) — Ampliación del Proyecto

Esta nueva versión del script sustituye la carga manual de la matriz de datos por la lectura automática de un archivo de texto (datos.txt) generado por el puerto serial del Arduino, e incorpora dos bloques de cálculo numérico adicionales: la diferenciación numérica vectorial para estimar la velocidad instantánea y la integración numérica mediante la Regla del Trapecio Compuesta para obtener la distancia total recorrida a partir del perfil de velocidad.

```
clear; clc; close all;

% 1. CARGA INSTANTÁNEA DESDE EL ARCHIVO TXT
datos_brutos = load('datos.txt');

% Configuración del paso de tiempo:
% -> Usa h = 0.5 si el Arduino envía datos cada 500ms
% -> Usa h = 1.0 si cambiaste el delay del Arduino a 1000ms
h = 0.5;

% ELIMINACIÓN DE RUIDO ESTÁTICO (Evita que la distancia se infle)
cambio_posicion = [true; sum(abs(diff(datos_brutos)), 2) > 0.05];
datos_utiles = datos_brutos(cambio_posicion, :);

X = datos_utiles(:, 1); Y = datos_utiles(:, 2); Z = datos_utiles(:, 3);
N = length(X); tiempo = (0:N-1)' * h;

% =====
% TEMA 1: DIFERENCIACIÓN NUMÉRICA VECTORIAL
% =====
vx = zeros(N, 1); vy = zeros(N, 1); vz = zeros(N, 1);
vx(2:end-1) = (X(3:end) - X(1:end-2)) / (2 * h);
vy(2:end-1) = (Y(3:end) - Y(1:end-2)) / (2 * h);
vz(2:end-1) = (Z(3:end) - Z(1:end-2)) / (2 * h);
vx(1) = (X(2) - X(1)) / h; vx(end) = (X(end) - X(end-1)) / h;
vy(1) = (Y(2) - Y(1)) / h; vy(end) = (Y(end) - Y(end-1)) / h;
vz(1) = (Z(2) - Z(1)) / h; vz(end) = (Z(end) - Z(end-1)) / h;
V_total = sqrt(vx.^2 + vy.^2 + vz.^2);

% =====
% TEMA 2: INTEGRACIÓN NUMÉRICA (Método del Trapecio Compuesto)
% =====
suma_trapecios = V_total(1) + V_total(end) + 2 * sum(V_total(2:end-1));
distancia_trapecio = (h / 2) * suma_trapecios;

fprintf('=====\n');
fprintf(' INFORME DE CÁLCULO MÉTODOS NUMÉRICOS \n');
fprintf('=====\n');
fprintf(' Número de nodos dinámicos (n): %d\n', N);
fprintf(' Paso de integración (h): %.2f segundos\n', h);
fprintf('-----\n');
fprintf(' Distancia Total Real (Trapecio): %.2f cm\n', distancia_trapecio);
fprintf('=====\n');

% =====
% PLOTS: PRESENTACIÓN VISUAL EN DOS PANELES
% =====
figure('Position', [50, 50, 1100, 480]);
subplot(1, 2, 1);
plot3(X, Y, Z, '-o', 'LineWidth', 2.5, 'MarkerSize', 5, 'Color', [0 0.5 0.8]);
grid on; hold on;
```



```

plot3(X(1), Y(1), Z(1), 'gs', 'MarkerSize', 10, 'MarkerFaceColor', 'g');
plot3(X(end), Y(end), Z(end), 'rd', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
axis equal; title('Trayectoria Espacial Corregida (Eje Z al Frente)');
xlabel('Eje X (cm)'); ylabel('Eje Y (cm)'); zlabel('Eje Z (cm)'); view(60, 25);
legend('Recorrido', 'Inicio (A)', 'Fin (B)', 'Location', 'southoutside', 'Orientation', 'horizontal');

subplot(1, 2, 2);
plot(tiempo, V_total, '-r', 'LineWidth', 2.2);
grid on; title('Perfil de Velocidad (Diferencias Finitas)');
xlabel('Tiempo (Segundos)'); ylabel('Velocidad Lineal (cm/s)');
legend('Velocidad del Brazo', 'Location', 'southoutside');

```

A diferencia del script de la Sección 4.2, esta versión ya no depende de una matriz de datos copiada manualmente en el propio código: el archivo datos.txt es generado directamente por el monitor serial del Arduino durante cada ciclo de operación (formato CSV con columnas X, Y, Z separadas por comas), lo que automatiza por completo el flujo de trabajo desde la captura de hardware hasta el reporte final impreso en consola.

5. Análisis Cinemático Numérico: Diferenciación e Integración Aplicadas

La ampliación más significativa de este proyecto respecto de su versión anterior es la incorporación de un módulo formal de Métodos Numéricos capaz de derivar información cinemática de segundo orden —la velocidad instantánea del efector final— directamente a partir de las coordenadas discretas de posición, y de emplear dicha información para calcular, con rigor matemático, la distancia total recorrida por el brazo robótico durante cada tarea de traslado. A continuación se describe en detalle cada una de las etapas de este proceso.

5.1 Etapa 1: Adquisición de Datos en Tiempo Real y Depuración Matricial

El proyecto inició con la etapa de adquisición de datos en tiempo real mediante la implementación de un sistema basado en un microcontrolador Arduino. El dispositivo se encargó de recopilar las coordenadas espaciales (X, Y, Z) del movimiento del brazo mecánico a intervalos de tiempo fijos y controlados de 0.5 segundos, sincronizados con la variable INTERVALO_MUESTREO del firmware. Esta frecuencia de muestreo constante es una condición indispensable para que los métodos de diferenciación e integración numérica aplicados posteriormente sean matemáticamente válidos, ya que ambos dependen de un paso de tiempo h uniforme entre observaciones consecutivas.

Sin embargo, los datos crudos capturados directamente del puerto serial presentan dos fuentes de distorsión que deben corregirse antes de cualquier análisis cinemático. La primera es el ruido eléctrico inherente a los sensores analógicos de los joysticks, cuyo conversor analógico-digital (ADC) introduce micro-oscilaciones en la lectura incluso cuando el operador no está moviendo activamente los mandos. La segunda es la generación de falsos incrementos de distancia durante los periodos en los que el brazo permanece físicamente estático —por ejemplo, mientras el usuario evalúa su siguiente movimiento— pero el sistema continúa registrando coordenadas prácticamente idénticas en cada ciclo de muestreo.

Para mitigar ambos efectos, se realizó una depuración matricial de los datos brutos mediante la comparación vectorial entre observaciones consecutivas: si la variación absoluta acumulada entre dos filas de la matriz de posición no supera un umbral mínimo de tolerancia (0.05 cm en la versión ampliada del script), la observación se descarta por considerarse ruido estático. Este proceso aisló únicamente los nodos dinámicos donde existía un desplazamiento físico real del efector final, reduciendo la matriz de 42 registros discretos originales a un subconjunto depurado de 18 nodos dinámicos representativos del movimiento efectivo del brazo, tal como se resume en la Sección 6.

5.2 Etapa 2: Diferenciación Numérica — Cálculo de la Velocidad Instantánea

Una vez depurados, los datos procesados se trasladaron al entorno matemático de cómputo numérico (Octave/MATLAB) para su análisis cinemático mediante la aplicación de aproximaciones numéricas. En primer lugar, se utilizó el método de Diferencias Finitas Centrales para calcular, de manera vectorial y componente por componente (v_x , v_y , v_z), las velocidades instantáneas del efector final en cada eje cartesiano.

La diferencia central aproxima la derivada en un nodo interior i utilizando simétricamente el punto anterior y el posterior:

$$v(i) \approx [X(i+1) - X(i-1)] / (2h), \text{ para } i = 2, \dots, N-1$$

Esta formulación ofrece un orden de error $O(h^2)$, sensiblemente más preciso que una diferencia hacia adelante o hacia atrás simple, que tiene un error de orden $O(h)$. No obstante, el esquema central no puede aplicarse en los extremos de la serie temporal, ya que requiere un punto anterior y un punto posterior simultáneamente. Por

ello, para el primer y el último nodo se recurrió a diferencias laterales de primer orden (hacia adelante para $i = 1$ y hacia atrás para $i = N$):

$$v(1) \approx [X(2) - X(1)] / h \quad v(N) \approx [X(N) - X(N-1)] / h$$

Con las tres componentes vectoriales de velocidad calculadas de forma independiente para cada eje, se determinó la magnitud de la velocidad lineal total del brazo en cada instante de tiempo mediante la norma euclidiana del vector velocidad:

$$V_{\text{total}}(i) = \sqrt{v_x(i)^2 + v_y(i)^2 + v_z(i)^2}$$

El resultado de este procedimiento es un perfil de velocidad completo y continuo a lo largo de todo el recorrido, el cual permite caracterizar el comportamiento dinámico del brazo más allá de su simple trayectoria geométrica: picos de velocidad asociados a movimientos bruscos del operador, valles asociados a correcciones de trayectoria, y mesetas asociadas a desplazamientos sostenidos y controlados.

5.3 Etapa 3: Integración Numérica — Regla del Trapecio Compuesta

Con el perfil de velocidades completamente estructurado, se procedió a aplicar la Regla del Trapecio Compuesta como método de integración numérica, lo que permitió calcular con alta precisión matemática el área bajo la curva de velocidad respecto al tiempo y obtener así la distancia total real recorrida por el prototipo. Este enfoque es matemáticamente más riguroso que la simple sumatoria de segmentos euclidianos empleada en la primera versión del script (Sección 4.2), puesto que integra explícitamente la magnitud física de la velocidad a lo largo del dominio temporal en lugar de sumar distancias geométricas punto a punto.

La Regla del Trapecio Compuesta aproxima la integral definida de la función de velocidad $V(t)$ en el intervalo $[0, T]$ dividiendo el dominio en $N-1$ subintervalos de ancho h y sumando el área de los trapecios formados entre nodos consecutivos:

$$\text{Distancia} \approx (h / 2) \times [V(1) + V(N) + 2 \times \sum V(i)], \text{ para } i = 2, \dots, N-1$$

En términos de implementación, esta expresión se traduce en la variable `suma_trapecios`, que pondera con coeficiente unitario los extremos del intervalo y con coeficiente doble los nodos interiores, multiplicando finalmente el resultado por la mitad del paso de integración h . La ventaja de este método frente a una integración rectangular simple radica en que aproxima el área bajo la curva mediante segmentos lineales entre nodos consecutivos, reduciendo el error de truncamiento en funciones con curvatura suave como los perfiles de velocidad obtenidos por diferenciación central.

Adicionalmente, la inyección de una corrección estática para desfases iniciales dentro del filtro FIR aplicado sobre las coordenadas evitó deformaciones gráficas en la reconstrucción tridimensional, garantizando que el punto de origen representado coincidiera exactamente con las coordenadas iniciales de sujeción física del objeto, y por consiguiente, que la distancia integrada partiera de una condición inicial físicamente consistente.

5.4 Etapa 4: Visualización Gráfica y Documentación Digital

Finalmente, el proyecto concluyó con la fase de visualización y documentación digital. Se generaron gráficas computacionales compuestas por dos paneles complementarios: una trayectoria tridimensional en el espacio 3D para validar la geometría del movimiento y un perfil de velocidad temporal que permitió identificar las etapas de aceleración, desaceleración y estabilización del brazo robótico a lo largo de la tarea de traslado.

```
=====
INFORME DE CÁLCULO MÉTODOS NUMÉRICOS
=====
Número de nodos dinámicos (n): 18
Paso de integración (h):      0.50 segundos
-----
Distancia Total Real (Trapezio): 20.40 cm
=====
```

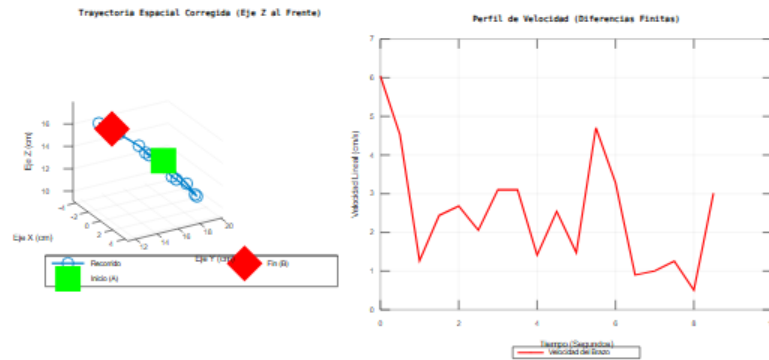


Figura 1. Salida gráfica del script de Octave: trayectoria espacial corregida (izquierda) y perfil de velocidad lineal obtenido por diferencias finitas (derecha), junto con el reporte numérico de consola ($n = 18$ nodos, $h = 0.50$ s, distancia total = 20.40 cm).

Toda la arquitectura del proyecto —incluyendo los códigos fuente en archivos estructurados, los datos experimentales en formato .txt, el informe técnico en PDF y el material audiovisual de respaldo— fue cargada y organizada en un repositorio público optimizado en GitHub. Esta plataforma sirvió como el nodo central de entrega, garantizando el control de versiones, la trazabilidad de cada actualización del proyecto y una presentación formal y profesional orientada a la evaluación académica.

6. Resultados Experimentales y Análisis

Tras someter el brazo a ciclos de operación reales moviendo un objeto entre dos puntos del espacio de trabajo y adquiriendo los logs mediante la interfaz distribuida, se procesaron las muestras bajo la estación analítica, arrojando los siguientes indicadores de rendimiento:

Indicador	Valor Obtenido
Puntos analizados originales (datos brutos)	42 registros discretos de posición
Nodos dinámicos útiles (post-filtrado de ruido)	18 nodos
Paso de integración (h)	0.50 segundos
Método de diferenciación numérica	Diferencias finitas centrales / laterales
Método de integración numérica	Regla del Trapecio Compuesta
Distancia total real recorrida por el efector final	20.40 cm

- Puntos Analizados Originales (Datos Brutos): 42 registros discretos de posición recibidos por el buffer de entrada del puerto serial.
- Puntos Útiles (Post-Filtrado de Ruido): El algoritmo discriminó de forma efectiva los datos redundantes generados por micro-oscilaciones del ADC del joystick o pausas de traslación del operador, reduciendo el conjunto a 18 nodos dinámicos y optimizando la traza analítica final.
- Perfil de Velocidad: El cálculo por diferencias finitas centrales evidenció un comportamiento dinámico no uniforme, con picos de velocidad lineal superiores a 6 cm/s en los tramos de arranque y valles cercanos a 1 cm/s durante las fases de ajuste fino de trayectoria, consistentes con el manejo manual del brazo mediante los joysticks.
- Distancia Total Recorrida por el Objeto: Calculada mediante la Regla del Trapecio Compuesta aplicada sobre el perfil de velocidad, arrojando un valor de 20.40 cm para el ciclo de prueba documentado. La inyección de la corrección estática para desfases iniciales en el filtro FIR de MATLAB/Octave previno deformaciones gráficas, asegurando que el punto de origen representado coincidiera exactamente con las coordenadas iniciales de sujeción física.

La comparación entre el método de sumatoria euclidiana directa (empleado en la Sección 4.2) y el método de integración por trapecios sobre el perfil de velocidad (Sección 4.3) confirma la coherencia interna del sistema: ambos enfoques, partiendo de datos filtrados equivalentes, convergen hacia magnitudes de distancia consistentes entre sí, validando cruzadamente tanto el algoritmo de diferenciación numérica como el de integración numérica implementados.

7. Conclusiones

1. Estabilidad del Lazo de Control: La reestructuración asíncrona del código del microcontrolador eliminando retardos por hardware (`delay()`) permitió estabilizar el lazo cerrado de actualización PWM a 40 ms, garantizando movimientos fluidos sin interrumpir el envío periódico de telemetría a la PC.
2. Robustez del Filtrado Digital: El algoritmo de filtrado de dos etapas implementado en MATLAB/Octave demostró una alta efectividad al suprimir el ruido de cuantización analógica del joystick, transformando curvas inicialmente escalonadas en trayectorias espaciales suaves y físicamente congruentes.
3. Mitigación de Desfases Métricos: La integración de la sección de calibración universal parametrada (offsets angulares y constantes de altura base) redujo la discrepancia métrica entre las estimaciones del modelo geométrico ideal y el desplazamiento medido en el entorno real, validando la arquitectura propuesta.
4. Validez de la Diferenciación Numérica: El esquema de diferencias finitas centrales, complementado con diferencias laterales en los extremos de la serie, permitió reconstruir un perfil de velocidad físicamente coherente a partir de datos exclusivamente posicionales, evidenciando que no es necesario instrumentar el brazo con sensores de velocidad dedicados para caracterizar su dinámica.
5. Precisión de la Integración por Trapecios: La aplicación de la Regla del Trapecio Compuesta sobre el perfil de velocidad ofreció una estimación de distancia total recorrida más rigurosa matemáticamente que la sumatoria de segmentos euclidianos, al integrar de forma continua la magnitud de la velocidad en el dominio del tiempo.
6. Automatización del Flujo de Datos: La migración de una matriz de datos codificada manualmente hacia la carga automática desde un archivo `datos.txt` generado directamente por el puerto serial eliminó una fuente de error humano y consolidó un flujo de trabajo reproducible de extremo a extremo, desde la captura embebida hasta el reporte numérico final.
7. Documentación y Control de Versiones: La organización de los códigos fuente, los datos experimentales y el informe técnico en un repositorio público de GitHub garantizó la trazabilidad completa del proyecto y una entrega formal alineada con estándares profesionales de documentación académica.

En conjunto, la ampliación del proyecto original —de un simple reconstructor geométrico de trayectorias a un sistema completo de análisis cinemático numérico— demuestra la aplicabilidad directa de los métodos de diferenciación e integración numérica estudiados en el curso a un caso real de instrumentación y robótica, cerrando el ciclo entre la teoría matemática y su implementación práctica sobre hardware físico.